

JPFFS — Documentação técnica do sistema

Campeonato JPFFS + Rachão + Copa Hendor + Gravação de Lances

Referência de desenvolvimento · criada em 05/09/2026 · atualizada em 02/10/2026 (cabeçalho ao vivo com câmeras, Galeria por lance, correções do Rachão) · cobre o que está no ar hoje

Documento voltado para quem mexe no código. Descreve arquitetura, modelo de dados, regras de negócio e operação. Para o comportamento funcional da Gravação de Lances do ponto de vista do usuário, ver o documento irmão [Documentacao-Gravacao-Lances-JPFFS.md](#).

Sumário

1. Visão geral e stack
 2. Arquitetura
 3. Modelo de dados
 4. Autenticação e autorização
 5. Ciclo de vida dos dados no cliente (`App.jsx`)
 6. Módulos `core/` (lógica pura)
 7. As telas
 8. Gravação de Lances (resumo técnico)
 9. Migrações SQL e infraestrutura Supabase
 10. Deploy e operação
 11. Convenções de desenvolvimento
 12. Limitações conhecidas e dívidas técnicas
-

1. Visão geral e stack

O JPFFS é um app de página única (SPA) para gerir uma pelada semanal: três módulos principais — **Campeonato** (rodadas com sorteio equilibrado de times, súmula, classificação, disciplina) e **Rachão** (fila por ordem de chegada, "quem vence fica"), a **Copa Hendor de Penalidades** (mata-mata de duplas em cobranças de pênalti, com chaveamento e disputa chute a chute) — mais o **Sistema de Gravação de Lances** (replay automático de gols usando celulares como câmeras).

Roda no celular, à beira da quadra. Feito para funcionar bem em tela pequena e tolerar conexão ruim.

Camada	Tecnologia
Front-end	React 18 + Vite 5, sem TypeScript
Estilo	Utilitárias de layout escritas à mão em <code>src/estilo.css</code> + <code>style</code> inline; tokens em <code>src/theme.js</code> . Sem Tailwind, sem CSS-in-JS.
Back-end	Supabase — Postgres (dados), Auth (login), Storage (fotos e vídeos), Realtime (sync e sinal de câmera), Edge Functions + Cron (limpeza de vídeo). Não há servidor próprio.
Hospedagem	Netlify — build automático a cada push em <code>main</code>
Offline	Service worker (<code>public/sw.js</code>) + <code>manifest.webmanifest</code> (instalável como PWA)
Dependências de runtime	<code>react</code> , <code>react-dom</code> , <code>@supabase/supabase-js</code> . Só isso.

Node 18+. Scripts: `npm run dev` (Vite, porta 5173), `npm run build` (→ `dist/`), `npm run preview` .

Por que a arquitetura é assim

- **Um `core/` puro e testável.** Toda regra de negócio (pontuação, disciplina, classificação, sorteio, fila do Rachão, Copa Hendor) vive em módulos sem React, que recebem dados e devolvem dados. Facilita raciocinar e testar isolado.
 - **Um único ponto de I/O.** `src/core/repositorio.js` é o único arquivo que fala com o Supabase. Trocar de back-end um dia é mexer só ali.
 - **Isolamento da Gravação de Lances.** Todo o código de câmera/vídeo é embrulhado em `<LimiteErro>` + `try/catch` . Se o Realtime cair, a súmula, a fila e o resultado do jogo continuam funcionando.
-

2. Arquitetura



O estado inteiro do Campeonato é um único JSON (base), carregado na inicialização, mantido em memória (`useState` no `App.jsx`), e regravado inteiro (debounce de 250 ms) a cada alteração feita por um organizador. Não há queries granulares — é "carrega tudo, salva tudo". Funciona porque o JSON é pequeno (dezenas de KB) e os escritores são poucos (só organizadores).

O Rachão não é persistido no servidor. Vive só no `localStorage` do aparelho e dura o dia. A única ponte com o Campeonato é o array `rodada.ordemChegada`.

3. Modelo de dados

3.1. Tabela `base` — o estado do Campeonato

Linha única, `id = 1`. Colunas: `dados jsonb`, `atualizado_em timestamptz`, `atualizado_por text` (e-mail de quem gravou por último — usado no aviso de "Atualizado por ..." via Realtime).

Shape de `dados` (ver `src/data/baseOficial.js` para o estado inicial e `migrarBase()` para o saneamento/migração no carregamento):

```

{
  "versao": 6,
  "campeonato": "Campeonato JPFFS",
  "temporada": 2026,

  "config": { /* ver CONFIG_PADRAO em core/regras.js – todas as regras */ },

  "jogadores": [
    {
      "id": "joao-vitor", // slug do nome, ou id() aleatório p/ novos
      "nome": "João Vitor",
      "posicao": "LINHA" | "GOLEIRO",
      "ativo": true, // false = fora da chamada, mantém histórico
      "convidado": false, // true = joga mas fica fora da classificação
      "estrelasIniciais": 1, // §11º – todo mundo entra com 1★
      "pendenciaFinanceira": false, // marca o $; nunca desconta ponto
      "pendenciaEfeito": "aviso", // só vale com o $ ligado: "aviso" | "sorteio" | "total" (ver
6.1)
      "pontuacaoPendente": false, // idem
      "posicaoInferida": false, // goleiro deduzido do ícone da tabela oficial
      "fotoUrl": "https://..." // opcional; bucket "avatares"
    }
  ],

  "rodadas": [
    {
      "id": "ab12cd34",
      "numero": 22,
      "data": "2026-09-06",
      "status": "aberta" | "fechada",
      "configSnapshot": { /* cfg congelado na abertura da rodada */ },

      "presencas": { "<jid>": "ausente" | "presente" | "atrasado" },
      "ordemChegada": ["<jid>", "..."], // ordem em que marcaram presença; ponte c/ Rachão

      "times": [
        {
          "id": "...", "partida": 1,
          "cor": "AMARELO" | "AZUL", "chave": "amarelo" | "azul",
          "seed": 123456, "extra": false,
          "jogadores": [
            { "jogadorId": "...", "estrelaNoSorteio": 4, "atuaComoGoleiro": false }
          ],
          "vagasAbertas": ["GOLEIRO", "LINHA"] // papéis ainda não preenchidos
        }
      ],

      "jogos": [
        {
          "id": "...", "numero": 1,
          "timeA": "<team id>", "timeB": "<team id>",
          "encerrado": false,
          "placarManual": null | { "A": 3, "B": 2 },
          "golsContraA": 0, "golsContraB": 0,
          "golsNaoComputadosA": 0, "golsNaoComputadosB": 0,
          "soCartoes": ["<jid>"], // entrou só p/ completar (§10º): não pontua, nem cartão
          "completaTime": [], // legado; unido a soCartoes no cálculo
          "eventos": { "<jid>": { "gols": 1, "assistencias": 0, "ca": 0, "cv": 0, "cz": 0 } }
        }
      ],

      "ajustes": [ { "id": "...", "jogadorId": "...", "valor": -1, "motivo": "..." } ],
      "sorteioRascunho": null | { /* prévia antes de "Gravar partidas" */ }
    }
  ]
}

```

```

    }
  ],

  "restricoes": [ { "id": "...", "a": "<jid>", "b": "<jid>", "tipo": "juntos" | "separados" } ],

  "copas": [ // Copa Hendor de Penalidades (ver 6.6)
    {
      "id": "hendor-2026", "tipo": "hendor", "nome": "Copa Hendor de Penalidades", "ano": 2026,
      "fases": [ { "id": "oitavas" | "quartas" | "semis" | "final", "nome": "Semifinais", "data":
"2026-10-31" } ],
      "partidas": [
        {
          "id": "s1", "fase": "semis", "rotulo": "Semifinal 1",
          "duplaA": { "jogadores": [<jid>, <jid>], "subs": [ { "id": "...", "sai": "<jid>", "entra":
<jid>", "motivo": "Ausente" } ] },
          "duplaB": { /* sem "jogadores" = montada a partir do vencedor da partida de origem */ },
          "origem": { "A": { "de": "q1", "tipo": "vencedor" }, "B": { "de": "q2", "tipo": "vencedor" }
        },
        "placarManual": { "A": 3, "B": 2 }, // fases já jogadas, só com o placar final
        "wo": "A" | "B", // lado que venceu por W.O. (Art. 54)
        "disputa": {
          "moeda": { "vencedor": "A", "escolha": "bater" | "defender" },
          "ordem": { "A": { "cobradores": [<jid>, <jid>], "defensores": [<jid>, <jid>] },
"B": { /* idem */ } },
          "chutes": [ { "lado": "A", "cobrador": "<jid>", "defensor": "<jid>", "fase": "regular" |
"alternada", "resultado": "gol" | "defendeu" } ],
          "lesionados": [<jid>"]
        }
      ]
    },
    ],
    "penalidades": [ { "id": "...", "jogadorId": "<jid>", "valor": -5, "motivo": "Ausência – Quartas
1",
      "partidaId": "q1", "trocaId": "...", "semSubstituto": false, "marcouDevendo":
false } ]
  }
],

  "historicoInicial": {
    "rodadas": 21,
    "data": "2026-08-01",
    "descricao": "Classificação oficial consolidada após a 21ª rodada",
    "jogadores": {
      "<jid>": { "P": 65, "J": 21, "V": 13, "E": 6, "D": 2,
        "GP": 34, "GC": 14, "CA": 1, "CV": 1,
        "Pmais": 0, "Pmenos": 1, "gols": 28, "assistencias": 2 }
    }
  }
}

```

Notas:

- `historicoInicial` é o acumulado até a 21ª rodada (a base "de fábrica" no `baseOficial.js`, formato pipe-delimitado no topo do arquivo). A classificação exibida = `historicoInicial` + o que foi lançado nas `rodadas` do app.
- `cz` em `eventos` = "cartão azul" (advertência local); com `converterSegundoAmarelo`, `ca≥2` ou `ca≥1 && cz≥1` viram um vermelho no cálculo (`normalizarCartoes`).
- `placarManual` sobrescreve a soma automática dos gols individuais. `placarDe()` devolve `divergente: true` quando os dois não batem.

- `configSnapshot` / `configPadrao` : as regras da rodada são congeladas na abertura, então mudar uma regra depois não reescreve rodadas antigas.
- `copas` mora no mesmo JSON da `base` (sem tabela nova, sem SQL). Placar e vencedor de cada partida são **derivados** (`placarManual` , `wo` ou os `chutes`); nada disso é guardado pronto. `migrarBase` injeta a Copa 2026 (`data/copaHendor2026.js`) quando a base ainda não tem `copas` , e ela passa a ser gravada no primeiro save de um organizador. A Copa acontece em datas FIFA, sem rodada do Campeonato no mesmo dia, então não disputa a escrita da linha única com as rodadas.
- `pendenciaEfeito` (ver 6.1) só vale com `pendenciaFinanceira` ligada; sem ele, o padrão é "aviso" .

3.2. Tabela `perfis` — contas de acesso

Ver `supabase-migracoes/001-perfis-e-aprovacao.sql` .

Coluna	Tipo	Observação
<code>id</code>	<code>uuid</code> PK	FK → <code>auth.users(id)</code> on delete cascade
<code>email</code>	<code>text</code>	
<code>papel</code>	<code>text</code>	<code>jogador</code> <code>organizador</code> (default <code>jogador</code>)
<code>status</code>	<code>text</code>	<code>pendente</code> <code>aprovado</code> <code>recusado</code> (default <code>pendente</code>)
<code>nome</code> , <code>telefone</code>	<code>text</code>	vêm do <code>options.data</code> do <code>signUp</code> (migração 002)
<code>criado_em</code>	<code>timestampz</code>	
<code>decidido_em</code> , <code>decidido_por</code>	<code>timestampz</code> , <code>uuid</code>	quem aprovou/recusou

O client **nunca** faz `insert` em `perfis` . O trigger `ao_criar_usuario` (security definer) em `auth.users` cria a linha como `jogador` / `pendente` . É o que impede autopromoção.

3.3. Tabela `lances` — metadados dos cliques

Ver `supabase-migracoes/004-lances.sql` e `006-lances-prod.sql` .

Coluna	Tipo	Observação
<code>id</code>	<code>uuid</code> PK	<code>gen_random_uuid()</code>
<code>modalidade</code>	<code>text</code>	<code>campeonato</code> <code>rachao</code>
<code>partida_id</code>	<code>text</code>	<code>camp-<rodada>-<jogo></code> <code>rachao-<sessão></code> <code>teste-camera</code>
<code>partida_rotulo</code>	<code>text</code>	texto amigável p/ agrupar na Galeria (ex.: "Rodada 5 · Partida 2")
<code>tipo</code>	<code>text</code>	<code>gol</code> <code>lance</code>
<code>jogador_id</code> , <code>jogador_nome</code>	<code>text</code>	opcional
<code>angulo</code>	<code>int</code>	número da câmera (1, 2, ...), via Presence
<code>caminho_storage</code>	<code>text</code>	<code>lances/<partida_id>/<ts>-a<angulo>.<ext></code>
<code>formato</code>	<code>text</code>	<code>video/mp4</code> (iPhone) <code>video/webm</code> (Android)
<code>criado_por</code>	<code>uuid</code>	FK → <code>auth.users</code>

Coluna	Tipo	Observação
<code>criado_em</code>	<code>timestampz</code>	

Realtime habilitado (`alter publication supabase_realtime add table public.lances + replica identity full`) — a Galeria atualiza sozinha.

3.4. Storage

Bucket	Público?	Uso	Acesso
<code>avatares</code>	sim (marcado Public)	fotos de jogador	upload/update só organizador; leitura livre pela URL pública
<code>lances</code>	não	clipes de vídeo	tudo por <code>createSignedUrl</code> (link de 1 h); upload por qualquer aprovado; delete só organizador

3.5. `localStorage` (por aparelho, nunca sincroniza)

Chave	Conteúdo
<code>jpffs:rachao</code>	sessão do Rachão do dia + convidados + <code>ordemIdx</code> (ordem de chegada p/ exibição)
<code>jpffs:rachao:rascunho</code>	(novo) lista da chamada manual do Rachão em dias sem rodada, montada antes de "Abrir Rachão"; descartada ao abrir ou se for de um dia passado
<code>jpffs:backup</code>	última cópia da <code>base</code> gravada com sucesso (rede de segurança)
<code>jpffs:cam:<partidaId></code>	"câmeras ativas" lembradas p/ esta partida/rodada/dia
<code>jpffs:camera-device</code>	id aleatório do aparelho-câmera (estável entre sessões)
<code>jpffs:camera-orientacao</code>	<code>h</code> (horizontal, padrão) <code>v</code> (vertical)
<code>jpffs:copaVista</code>	<code>arvore</code> (padrão) <code>fases</code> — como o chaveamento da Copa é mostrado neste aparelho

4. Autenticação e autorização

4.1. Papéis

- **Organizador** = `perfil.papel === "organizador" && perfil.status === "aprovado"`. Vê e edita tudo.
- **Jogador aprovado** = `status === "aprovado" (papel jogador)`. Vê só as abas **Tabela** e **Lances**.
- **Pendente / recusado / sem perfil** → tela "Cadastro em análise" / "não aprovado" (`TelaAcesso.jsx`), sem acesso ao app.

`App.jsx` deriva `souOrganizador` / `souAprovado` do perfil e:

- filtra as abas da navegação;
- redireciona pra "tabela" se um não-organizador cair noutra aba;
- só busca a `base` **depois** que o perfil está aprovado (antes, a RLS bloqueia a leitura de qualquer jeito);
- só grava a `base` se `souOrganizador`.

4.2. RLS (a segurança de verdade)

O app só "busca e decide" — quem barra é o Postgres. Duas funções `security definer` furam a recursão de RLS sobre a própria `perfis`:

- `public.eh_aprovado()` → O `auth.uid()` tem `status = 'aprovado'` ?
- `public.eh_organizador_aprovado()` → ... e `papel = 'organizador'` ?

Ambas `revoke all from public` + `grant execute to authenticated`.

Policies (resumo):

Recurso	SELECT	INSERT/UPDATE/DELETE
<code>base</code>	<code>eh_aprovado()</code>	<code>eh_organizador_aprovado()</code>
<code>perfis</code>	própria linha, ou tudo se <code>eh_organizador_aprovado()</code>	UPDATE/DELETE só organizador; sem INSERT (só o trigger)
<code>lances</code>	<code>eh_aprovado()</code>	INSERT qualquer aprovado; DELETE só organizador
Storage <code>avatares</code>	<code>eh_aprovado()</code> (via RLS) + URL pública	escrita só organizador
Storage <code>lances</code>	<code>eh_aprovado()</code>	INSERT qualquer aprovado; DELETE só organizador

Pegadinha recorrente (já quebrou produção uma vez): tabela criada por SQL direto **não** ganha `GRANT` para `authenticated` sozinha (o Table Editor visual ganha). Toda tabela nova precisa de `grant select, insert, update, delete on public.<t> to authenticated;` explícito, **além** da RLS. Sem isso: "permission denied for table ..." mesmo com a policy certa.

4.3. Fluxo de cadastro

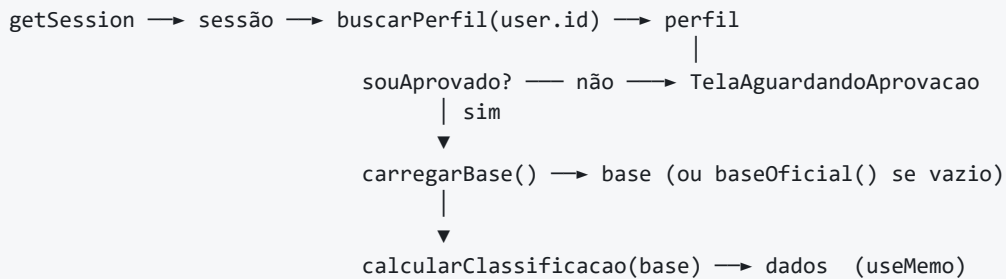
1. "Criar usuário" na tela de login → `supabase.auth.signUp({ email, password, options: { data: { nome, telefone } } })`.
2. Trigger cria `perfis` como `jogador / pendente`.
3. Com "**Confirm email**" **DESLIGADO** no Supabase (estado atual, recomendado), o `signUp` devolve sessão na hora → o app cai direto em "aguardando aprovação". O e-mail nativo do Supabase é pouco confiável (rate limit baixo, cai em spam), e a aprovação do organizador já é o portão.
4. Organizador aprova em **Ajustes** → **Cadastros de acesso** (`SecaoAprovacoes` em `TelaConfig.jsx`) → `decidirPerfil(id, "aprovado", meuId)`.
5. `App.jsx` re-busca o perfil ao voltar o foco da aba, então a pessoa entra sozinha assim que aprovada, sem deslogar.

Revogar acesso = `decidirPerfil(id, "recusado")` (botão "Bloquear"), reversível. **Não existe "excluir de vez" pelo app** — apagar só a linha de `perfis` deixava o login órfão e a pessoa num limbo permanente ("cadastro em análise", sem aparecer na lista do organizador). Apagar o login mesmo: painel Supabase → Authentication → Users → Delete user.

Histórico do bug e do fix: memória `jpffs-auth-cadastro` ; migrações 003 (policy de delete, hoje só usada como rede) e 005 (backfill de contas órfãs criadas enquanto o trigger da 001 ainda não existia — não deve mais ocorrer).

5. Ciclo de vida dos dados no cliente (App.jsx)

O `App.jsx` orquestra sessão → perfil → base, nesta ordem, cada um com sua tela de "Carregando...".



Sincronização da `base` (só p/ aprovados):

- **Ao ganhar foco / aba visível:** `carregarBase()` de novo (a menos que haja alteração local pendente de salvar — `salvandoPendenteRef`).
- **Realtime:** canal `base:realtime` escuta `UPDATE` em `public.base`. Chega uma versão nova → `migrarBase()` + `setBase()` + aviso "Atualizado por " se não foi este aparelho.
- **Salvar:** qualquer `setBase` de um organizador dispara, com debounce de 250 ms, `salvarBase(base)` → `UPDATE public.base SET dados = ... WHERE id = 1`. Também grava `jpffs:backup` no `localStorage` antes de tentar a rede, e tem 1 retry após 1,2 s.
- `pularProximoSalvar` evita o loop "recebi do Realtime → salvei → recebi de novo".

Modo câmera: `?camera=1` na URL → `MODO_CAMERA`. O app renderiza só um shell mínimo com a `TelaCamera`, sem navegação. Continua exigindo login aprovado.

Primeira tela: o estado `campeonato` (`null` | `"jpffs"` | `"hendor"`) decide o que renderizar. `null` mostra a `TelaEscolha` — sempre, a cada abertura (de propósito, não é lembrado); `"jpffs"` mostra as abas de sempre; `"hendor"` mostra a `TelaCopaHendor` com barra própria (Chaveamento · Resultados · Documentação). O botão "Trocar" do cabeçalho volta a `null`. Visitante aprovado só lê; quem grava é o organizador, como no resto do app.

Modo ensaio: `?simulacao=1` liga `SIMULACAO` (exportada de `repositorio.js`). `salvarBase` vira no-op (nem banco, nem `jpffs:backup`), o Realtime e a sincronização por foco são ignorados e uma faixa amarela avisa. Existe porque o `npm run dev` fala com o **mesmo Supabase de produção**.

6. Módulos `core/` (lógica pura)

6.1. `core/regras.js` — pontuação, disciplina, classificação, sorteio

O maior módulo. Tudo puro: recebe `base`, devolve dados.

`CONFIG_PADRAO` — todas as regras num objeto só (pontos por vitória/empate/ presença, teto por rodada, ciclos de cartão, atrasos para suspensão, tamanho de time, zona da Supercopa, critérios de desempate, pesos do motor de sorteio). Na UI (aba Ajustes) quase tudo aparece **bloqueado** — a edição real está desativada de propósito. Os campeões da Copa Hendor não são mais digitados: saem da final da Copa (`core/copaHendor.js`, seção 6.6).

Pontuação (`calcularEstatisticas`): $P = J \cdot 1 + V \cdot 3 + E \cdot 1 + D \cdot 0 + P^+ - P^-$, onde P^- = penalidade manual + pontos de atraso + penalidade de cartão amarelo (por ciclo fechado de 3) + penalidade de vermelho. O `teto (rodadasRealizadas · tetoPorRodada)` serve para o `aproveitamento %`.

Disciplina (`disciplinaAtrasos`): conta atrasos por jogador, com reset mensal (salvo "emenda" — atraso em mês novo mantém o contador). Níveis: 1º = alerta, 2º = cartão amarelo na classificação, 3º = perde o ponto de presença, 4º = suspenso da rodada (`atrasosParaSuspensao`).

Classificação (`calcularClassificacao`): ordena por `critériosDesempate` (pontos → vitórias → saldo → golsPro → cartões → alfabetica), separa linha × goleiro para os rankings de categoria, marca a **zona da Supercopa** (12 de linha + 2 goleiros, com os campeões da Copa Hendor — `campeoesHendor(base)` — entrando mesmo fora do corte), e deriva as **estrelas por posição na tabela** (1–3 → 5★, 4–6 → 4★, 7–9 → 3★, 10–14 → 2★, resto → 1★). Convidados ficam de fora.

Motor de sorteio (`sortearEquipes`): recebe as "entradas" (presentes aptos com estrelas/aproveitamento), distribui em `partidas·2` times, e roda uma **busca local** (`buscaLocal`, trocas 2-a-2 por até 40 passes) minimizando uma função de custo (`avaliarTimes`) que pondera:

Fator	Peso (<code>cfg.pesos</code>)
violação rígida (5★ demais num time, 2 goleiros juntos, restrição de par)	<code>rigida</code> (100000)
amplitude da média de estrelas entre times	<code>amplitude</code>
desvio-padrão das médias	<code>desvio</code>
distribuição por faixa de estrela	<code>faixa</code>
5★ espalhados entre partidas	<code>faixaPartida</code>
variância interna de cada time	<code>varianciaInterna</code>
repetição de duplas das últimas N rodadas	<code>repeticao</code>
aproveitamento % (se <code>usarAproveitamento</code>)	<code>aproveitamento</code>

Comparações são por **média de estrelas por jogador**, não soma bruta — assim a partida que sobra com menos gente não é tratada como desequilibrada só por ter menos jogadores. `seed` determinístico (`core/rng.js`): mesma seed → mesmo sorteio ("Repetir seed" na UI). Vagas que não fecham viram

`vagasAbertas` e são preenchidas depois, na etapa `Partidas`.

`sortearParcial` (em `TelaRodada.jsx`) resorteia **uma** partida sem remover quem já está nela.

Pendência financeira (\$). Cada jogador tem `pendenciaFinanceira` e, se ligada, um `pendenciaEfeito` (`efeitoPendencia(j)` devolve "aviso" quando não há efeito definido):

Efeito	Chamada do Campeonato	Sorteio	Rachão
aviso (padrão)	normal	entra	entra
sorteio	marca presença e entra na ordemChegada	fora (<code>poolsDoDia</code> o tira de <code>aptos</code> e o lista em <code>barradosPendencia</code>)	joga
total	botão travado	fora	fora (lista manual e "Adicionar à fila")

`barradoDoSorteio(j)` e `barradoDoRachao(j)` são as funções que a UI consulta. Nunca desconta ponto. O bloqueio vale dentro do app — não há trava no banco.

6.2. `core/rachao.js` — fila do Rachão (Art. 25º–30º do Estatuto)

Também puro: recebe uma "sessão" (o rachão de um dia), devolve uma sessão nova. **Nunca é persistido** — só vive no estado da tela.

Conceito: **uma quadra**, times Amarelo × Azul formados por **ordem de chegada** (sem balancear estrela), "quem vence fica". Fila de linha e fila de goleiros separadas. `linha` e `goleiros` guardam **todo mundo do dia** — sair da fila pra entrar em quadra não remove ninguém, só passa a contar como "em quadra".

Funções-chave: `criarSessao`, `iniciarPartida`, `marcarGol`, `encerrarPartida` (o coração das regras — empate na 1ª partida, corte do Art. 29º, "fila grande", par-ou-ímpar), `substituirLinha`, `removerJogador`, `inserirNaFila`, `reclassificarJogador`. O cabeçalho do arquivo documenta cada suposição de regra em detalhe — **leia antes de mexer**.

Correções de fim de partida: `aplicarDesfecho` guarda em `sessao.desfazer` uma foto da sessão **antes** do encerramento (1 nível — só a última partida). `reabrirUltimaPartida` restaura essa foto (quadra com o placar, fila de antes), preserva quem chegou/saiu depois e devolve `desfazer: null`. `encerrarManual(sessao, lado|null)` encerra escolhendo quem fica (ou "os dois saem"), passando pelo mesmo `aplicarDesfecho` (fila, goleiros e corte do Art. 29º seguem normais). Testes: `testes/rachao.teste.mjs`.

6.3. `core/lances.js` — buffer duplo de gravação

Motor de captura de clipe de **20s fixos** (`TOTAL_MS`). **Dois** `MediaRecorder` gravam o mesmo stream, defasados meio ciclo (`JANELA=17s` , `DEFASAGEM=8,5s`). No sinal, escolhe o gravador com mais história, deixa rodar `TOTAL - idade do buffer` (entre 3 e 12 s) e chama `stop()` — o Blob que sai é um arquivo **encerrado de verdade** pelo navegador (duração correta, sem buraco). Concatenar chunks de uma gravação em andamento **não funciona** (o navegador só finaliza os metadados no `stop()`).

- `abrirCamera()` — `getUserMedia` cru (1920×1080 ideal).

- `criarStreamVertical(streamRaw)` — desenha cada frame num `<canvas>` 720×1280 e devolve `canvas.captureStream()` + áudio. Usado só no **modo vertical**; gira 90° quando a câmera vem deitada e a tela está em pé (iPhone entrega a câmera deitada mesmo com o celular em pé). No **modo horizontal** grava o stream cru direto.
- `criarGravador(stream, { aoMudarEstado })` — os dois canais, a reciclagem agendada numa grade absoluta, `capturar(id)` (Promise), e `reiniciarGravadores()` (recupera de "tela preta" — quando os chunks param de chegar por > 3 s).

6.4. `core/exportacao.js` — saídas para download

CSV (classificação, súmula), texto de WhatsApp (escalação), e **imagens PNG geradas via** `<canvas>` (tabela de classificação, escalações da rodada). Nenhuma regra mora aqui, só formatação.

6.5. `core/repositorio.js` — I/O

Único ponto que fala com o Supabase. `carregarBase` / `salvarBase`, `buscarPerfil` / `listarPerfis` / `decidirPerfil`, `enviarFotoJogador`, `enviarLance` / `listarLances` / `urlAssinadaLance` / `excluirLance` / `tituloLance` / `nomeArquivoLance`, `migrarBase` (saneamento + defaults no carregamento, incluindo a injeção da Copa 2026 quando falta `copas`), `corrigirMojibake` (conserta texto com double-encoding de UTF-8 preso em registros antigos), `id()` (gerador de id curto).

`listarLances(filtro)` aceita `string` (= `partidaId`, `compat`), `{ partidaId }`, `{ partidaPrefixo }` (todas as partidas de uma rodada), `{ desde, ate }` (janela de tempo — o link "do dia" cobre Campeonato + Rachão) ou `{ modalidade }`.

`SIMULACAO` (flag do modo ensaio, seção 5) é lida uma vez, com guarda `typeof window` para o módulo poder ser importado fora do navegador (é o que permite rodar `testes/` no Node).

6.6. `core/copaHendor.js` — Copa Hendor de Penalidades (Arts. 41–56 e 85)

Só funções puras sobre `base.copas[]` (sem imports — `regras.js` importa este arquivo). Cada partida guarda as duplas (`jogadores` + `subs`), de onde vêm os lados (`origem`), o placar lançado à mão nas fases já jogadas (`placarManual`), o W.O. (`wo`) e a `disputa` chute a chute. Uma dupla sem `jogadores` é montada a partir do vencedor da partida de origem, e as substituições (Art. 55 §1) trocam quem saiu por quem entrou — então o chaveamento avança sozinho.

- **Sequência dos chutes** (`chuteEsperado`): 8 cobranças (X1 bate 2, Y1 bate 2, X2, Y2; cada cobrador contra os dois defensores adversários), depois **alternadas** (uma por dupla, em rodízio) até desempatar. Todos têm direito às 4 cobranças: a disputa só decide depois do 8º chute, ou ao fechar uma rodada de alternadas.
- **Lesão** (Art. 55 §3): o parceiro executa os chutes e defesas que faltam. **W.O.** (Art. 54): `wo` = lado vencedor.
- **Substitutos** (`substitutosPossiveis`, Art. 55 §1): `daFaseAnterior` (eliminados da fase anterior, por classificação), `outros` (qualquer outro jogador ativo fora da Copa — escolha do organizador) e `bloqueados` (com `$` em "sem sorteio"/"bloqueado", Arts. 42 e 85; "só avisar" continua elegível). Ficam de fora quem já joga a fase, quem já foi trocado por ausência, inativos e convidados. A regra de inadimplência repete `barradoDoSorteio` porque `copaHendor.js` não pode importar `regras.js` (import circular).

- **Desfazer troca / W.O.** (`desfazerUltimaTroca` , `desfazerWo`): só antes da disputa começar. Cada troca tem `id` e a penalidade `-5` dela guarda `trocaId` (e `marcouDevendo` , se foi a troca que ligou o \$), então o desfazer remove exatamente o que a troca criou e nada mais.
 - **Campeões** (`campeoesHendor`): dupla vencedora da final, que alimenta a zona da Supercopa.
 - **Penalidade `-5`** (Art. 55 §4): `copa.penalidades[]` , somada como desconto manual em `calcularEstatisticas` (a Copa roda em data FIFA, sem rodada do Campeonato, por isso não usa os ajustes da rodada).
 - `estatisticasJogadores` : gols e defesas por jogador (aba Resultados);
`faseAtual` / `statusDaFase` / `statusDaPartida` (`aguardando` → `pronta` → `em_andamento` → `encerrada`) alimentam a árvore e a tela de escolha.
 - Dados iniciais da Copa 2026 em `data/copaHendor2026.js` (oitavas e quartas com placar); `migrarBase` injeta a copa quando a base ainda não tem `copas` . Testes: `npx vite-node testes/copaHendor.teste.mjs` .
-

7. As telas

Cada arquivo em `src/telas/` é uma aba. `App.jsx` roteia por `aba` e passa `{ base, setBase, dados, cfg, avisar }`. A **primeira tela** (`TelaEscolha`, sempre exibida ao abrir) pergunta qual campeonato ver:

Campeonato JPFFS (as abas abaixo, como sempre) ou **Copa Hendor** (`telas/copa/`: Chaveamento — em **árvore** (`ArvoreChaveamento`, rola pro lado no celular e abre na fase atual) ou **por fase**, escolha lembrada no aparelho —, Resultados, Documentação; visitante só vê, organizador lança as cobranças, troca jogadores e dá W.O. direto nos cartões das partidas). O botão "Trocar" do cabeçalho volta à escolha. `?simulacao=1` na URL liga o **modo ensaio**: nada é gravado nem recebido em tempo real (útil porque o `npm run dev` usa o banco de produção).

Tela	Aba	Quem vê	O que faz
<code>TelaClassificacao</code>	Tabela	todos	Classificação geral, resultados por rodada, e a aba Documentação (regras do Estatuto, dentro do app). Exporta CSV/PNG.
<code>TelaRodada</code>	Rodada	organizador	Fluxo de 3 etapas: Presença (chamada, registra <code>ordemChegada</code> ; pendência <code>total</code> trava o botão e <code>sorteio</code> marca mas fica fora do sorteio) → Sorteio (motor de equilíbrio, ajuste fino arrastando, "Gravar partidas") → Partidas (súmulas ao vivo: gols, assistências, cartões, gol contra, gol não computado; encaixe de vagas abertas; ⇌ troca o jogador de qualquer vaga (leva o "só completando" \$10 se escolhido; apaga os lançamentos do que sai, com confirmação); ajustes <code>P+/P-</code> ; fechar rodada).
<code>TelaRachao</code>	Rachão	organizador	Abertura (puxa <code>ordemChegada</code> da rodada do dia ou chamada manual quando não há rodada) → quadra ao vivo, fila arrastável, próximos times, histórico do dia; reabrir a última partida e decidir na mão quem fica (seção 6.2). Fila, Goleiros e Adicionar à fila numa coluna centralizada de 540px. Estado no <code>localStorage</code> . Jogador com pendência <code>total</code> não entra na lista nem na fila.
<code>TelaElenco</code>	Elenco	organizador	Cadastro/edição de jogadores, foto, posição, flags (ativo, convidado, pendência financeira com efeito : só avisar / sem sorteio / bloqueado), importar CSV/JSON.
<code>TelaConfig</code>	Ajustes	organizador	Cadastros de acesso (aprovar/bloquear), histórico de rodadas (reabrir recalcula), regras (quase tudo bloqueado), export/import da base (JSON), restaurar padrão / base oficial.
<code>TelaGaleria</code>	Lances	aprovados	Clipes agrupados por partida e, dentro dela, por lance : os ângulos do mesmo lance viram um cartão só, lado a lado, com "Ver juntos" (player com os vídeos lado a lado). Filtros modalidade/tipo/jogador; ver, baixar (todos), apagar (só organizador). Atualiza via Realtime.
<code>TelaCamera</code>	— (link <code>?camera=1</code>)	aprovados	Vira o aparelho numa câmera: liga a câmera, entra no canal, Presence numera o ângulo, grava no sinal, sobe pro bucket <code>lances</code> . "Modo gravação" = tela cheia + Wake Lock.

Componentes compartilhados em `src/components/ui.jsx` (`Botao`, `Painel`, `Campo`, `Segmento`, `SecaoRecolhivel`, `CampoBusca`, `Contador`, `Estrelas`, `IconeGoleiro`, `AvatarJogador`, ...) e ícones SVG inline em `src/components/icones.jsx` (baseados no Lucide). `LimiteErro` é o error boundary que embrulha os

blocos de Lances.

Cabeçalho ao vivo. No Rachão e na Gestão da Rodada, título, câmeras e cronômetro ficam numa faixa só que gruda no topo ao rolar (`CabecalhoAoVivo` , CSS `.cabecalho-ao-vivo` em `estilo.css` , `position: sticky` logo abaixo da barra do app — a altura dela é medida no `App.jsx` e publicada em `--altura-cabecalho-app`). A barra de câmeras é compacta (status/desligar · Gravar lance · copiar link) e a classificação do lance / "guardar vídeo do gol?" abre logo abaixo, dentro do próprio cabeçalho.

7.1. Chamada manual do Rachão (dias sem rodada)

Adicionada em setembro/2026. Quando **não há rodada do Campeonato** na data escolhida, a abertura do Rachão mostra a seção "**Lista do dia**": grade do elenco com busca (toque = ordem de chegada), lista numerada removível, e campo de convidado do dia (fica só na sessão do Rachão, não encosta no elenco). O rascunho é salvo em `localStorage` (`jpffs:rachao:rascunho`) e sobrevive a F5/lock; é descartado ao abrir o Rachão ou quando é de um dia passado. Com rodada na data, nada muda — continua puxando `ordemChegada` do Campeonato.

8. Gravação de Lances (resumo técnico)

Documento funcional completo: [Documentacao-Gravacao-Lances-JPFFS.md](#) . Resumo para desenvolvimento:

- **Sem backend.** O clipe é fechado pelo navegador do celular-câmera (buffer duplo, seção 6.3). iPhone grava MP4, Android WebM; cada um toca direto no `<video>` , sem conversão.
- **Sinal em 2 fases via Supabase Realtime**, canal `lances:<canalId>` :
- `disparo` (broadcast) → todas as câmeras ativas capturam o "depois" (3–12 s, fecha 20 s). Carrega `{ id, modalidade, partidaId, partidaRotulo }` .
- `decisao` (broadcast) → `salvar` (com `tipo` e `jogadorNome`) ou `descartar` .
- `presence` → numera os ângulos (1, 2, ...) na ordem de conexão.
- Um novo `disparo` durante o "depois" de outro é ignorado (trava local).
- **canalId é do DIA:** `dia-<AAAA-MM-DD>` (= `rodada.data` / `sessao.data`). O **mesmo link de câmera cobre o Campeonato e o Rachão** do dia — as partidas rolam uma de cada vez no mesmo campo. A modalidade/partida de cada clipe vem no `disparo` , então a Galeria separa tudo sozinha. Links antigos (`camp-<rodada>` , `camp-<rodada>-<jogo>` , `rachao-<sessão>`) ainda funcionam.
- **partida_id do clipe** (etiqueta, ≠ canal): `camp-<rodada>-<jogo>` / `rachao-<sessão>` / `teste-camera` .
- **Gatilhos:** `GatilhoLances` (Rachão — botão único, tipo escolhido depois) e `GatilhoLancesCampeonato` (`forwardRef` , expõe `golMarcado(...)` — o "+" do jogador na súmula é que dispara; "Gravar lance" separado para não-gols, com seletor de partida quando há mais de uma aberta). Ambos em `<LimiteErro>` . A parte visual é compartilhada em `BarraCameras` e mora no `CabecalhoAoVivo` (seção 7).
- **Campeonato:** o estado das câmeras vive em `RodadaEmAndamento` (`TelaRodada` , `key={rodada.id}`), não na aba Partidas — o gatilho fica montado em qualquer etapa da rodada e a súmula continua chamando `golMarcado` pelo ref.
- **Câmeras ativas** lembradas por aparelho (`jpffs:cam:<chave>`), some só ao tocar no status "Ao vivo ✕".
- **Galeria por lance:** `juntarAngulos` (`ListaClipes`) junta num cartão os clipes da mesma partida + tipo + jogador que subiram até 90 s um do outro — nunca dois do mesmo ângulo no mesmo cartão (ângulo repetido vira cartão próprio, pra não esconder nada).
- **Tela preta:** a `TelaCamera` detecta quando os pedaços de vídeo param de chegar (> 3 s — tela apagou / app foi pro fundo) e mostra "TELA TRAVOU — toque para retomar", que reinicia só os gravadores, sem derrubar câmera nem canal.
- **Orientação** escolhida pelo operador na `TelaCamera` (`jpffs:camera-orientacao`): horizontal (padrão, pega mais campo) grava cru; vertical passa pelo canvas.
- **Vídeo:** 720p comprimido (~1,8 Mbps, ~5 MB / 20 s) — imposto pelo limite de 1 GB de storage do plano free.
- **Retenção:** 5 dias corridos + trava de espaço (ver seção 9.3).

8.1. Pendências da feature (feedback do 1º teste em campo, 03/09/2026)

- **Mesmo ângulo gravado 2×** — a atribuição de ângulo por `Presence` reembalha em reconexão, e/ou o `disparo` é processado 2×. Falta dedup por `cid` + opção de fixar o ângulo na mão.
- ~~Duração não fecha 20 s exatos~~ — **resolvido** em 19/09 (duração fixa, seção 6.3); falta confirmar em campo.

- **Falta teste em campo / no celular** de: Campeonato ponta-a-ponta, Galeria ao vivo (agora com ângulos lado a lado), cabeçalho congelado, retomada de "tela travou".
 - **Cron de limpeza:** consertado em 19/09 (ver 9.3) — conferir que responde 200 e que os cliques antigos sumiram.
-

9. Migrações SQL e infraestrutura Supabase

9.1. Pasta `supabase-migracoes/`

Não é o Supabase CLI. São arquivos numerados para colar no **SQL Editor** do painel, em ordem. Todos idempotentes.

Arquivo	O que faz
<code>001-perfis-e-aprovacao.sql</code>	tabela <code>perfis</code> , trigger de conta nova, funções <code>eh_aprovado</code> / <code>eh_organizador_aprovado</code> , reaperta RLS de <code>base</code> e do bucket <code>avatares</code> (leitura passa a exigir login aprovado)
<code>002-nome-telefone.sql</code>	colunas <code>nome</code> / <code>telefone</code> em <code>perfis</code> ; trigger passa a gravá-las do <code>raw_user_meta_data</code>
<code>003-excluir-perfis.sql</code>	policy + grant de DELETE em <code>perfis</code> p/ organizador (hoje usada só como rede — o app não deleta mais)
<code>004-lances.sql</code>	tabela <code>lances</code> + policies + bucket privado <code>lances</code> + policies de storage
<code>005-backfill-perfis.sql</code>	cria <code>perfis</code> p/ contas órfãs (criadas antes do trigger da 001 existir). Já rodado; não deve mais ser necessário
<code>006-lances-prod.sql</code>	Realtime em <code>lances</code> , coluna <code>partida_rotulo</code> , função <code>uso_bucket_lances()</code> , <code>pg_net</code> + <code>pg_cron</code> , índices, e o passo-a-passo do Cron
<code>007-lances-service-role.sql</code>	grant select, delete on <code>lances</code> to <code>service_role</code> — sem isso a Edge Function de limpeza dava <code>permission denied</code>
<code>limpar-lances/index.ts</code>	Edge Function da limpeza (ver 9.3)

Ao adicionar uma tabela nova: **lembrar do** `GRANT ... to authenticated` (seção 4.2) e, se for consumida por Realtime, `add table` na publication + `replica identity full` .

9.2. Ordem obrigatória em mudanças com SQL

SQL primeiro (rodado e confirmado no painel), **depois** o código que depende dele. Nunca subir código que espera uma coluna/policy/tabela que ainda não existe — isso já travou o login do organizador uma vez.

Funções `security definer` (ex.: `eh_aprovado` , `uso_bucket_lances`): sempre `revoke all from public` + `grant execute` explícito só pro papel certo.

9.3. Limpeza automática de vídeo

Edge Function `limpar-lances` (`supabase-migracoes/limpar-lances/index.ts`) + **Cron Job** a cada 30 min (`*/30 * * * *`). Responde 200 na hora e limpa em segundo plano (`EdgeRuntime.waitUntil`).

1. Apaga clipes com `criado_em` > 5 dias.
2. **Trava de espaço:** se `uso_bucket_lances()` > 850 MB, apaga os mais antigos até baixar de ~650 MB (plano free = 1 GB; sem isso o upload falharia no meio de uma rodada).

Sempre apaga o arquivo no Storage **antes** da linha da tabela — se o storage falhar, a linha fica e a próxima execução tenta de novo.

Pegadinhas (consertadas em 19/09/2026 — a limpeza nunca tinha funcionado antes):

- A função tem `verify_jwt = true` → o Cron precisa mandar `Authorization: Bearer <anon JWT>` no `net.http_post`. Com `headers := '{}'` toda chamada voltava **401**.
- O `service_role` precisa de `select, delete` em `lances` (migração 007).

Conferir execuções — `cron.job_run_details` mostrar "succeeded" só quer dizer que o `pg_net` enfileirou a requisição; o status HTTP de verdade está em `net._http_response`: `select * from cron.job_run_details order by start_time desc limit 5; select id, status_code, created from net._http_response order by created desc limit 5;`

9.4. Supabase MCP

`.mcp.json` (escopo "project", não versionado — está no `.gitignore` implícito / não trackeado). Autenticado via OAuth pelo usuário numa sessão. Escrita de DDL pelo MCP é bloqueada pelo classificador do Claude Code → **DDL vai pelo painel**. Numa sessão nova, se as tools do Supabase não aparecerem, rodar `/mcp`.

10. Deploy e operação

- **App:** Netlify conectado ao repo GitHub. Cada `git push` em `main` dispara um build (`npm run build`, `publish dist/`). Config em `netlify.toml` (SPA redirect `/* → /index.html`, `sw.js` com `Cache-Control: no-cache`).
- `dist/` **é versionado no git** (não está no `.gitignore`). Rodar `npm run build` local gera arquivos com hash novo em `dist/assets/` — **sempre** `git checkout -- dist/` e apagar os `?? dist/assets/index-*.js` novos antes de commitar. O Netlify builda a partir do fonte, não usa o `dist/` commitado.
- **Fluxo de commit:** branch `feature/<nome>` → commit → merge `--no-ff` em `main` → **confirmar com o usuário antes do** `git push origin main` (é produção com usuários reais).
- **Supabase:** projeto `njwhdzntwzgoxcyhhwod` . Chave pública (publishable) no código (`src/supabase.js`) — sem risco, a proteção é a RLS. "Confirm email" DESLIGADO.
- **Backup:** o organizador deve exportar o JSON da base (Ajustes → Exportar Dados) periodicamente. É o backup portátil e o jeito de levar pra outro aparelho. O `localStorage` (`jpffs:backup`) é só rede de segurança local.

10.1. Testar no celular (câmera exige HTTPS)

`npm run dev` + Cloudflare Tunnel (`cloudflared`, já em `C:\Users\Miran\cloudflared\cloudflared.exe`):
`cloudflared tunnel --url http://localhost:5173` → abre a URL `*.trycloudflare.com` no celular.
`vite.config.js` já tem `host: true` + `allowedHosts: ['.trycloudflare.com']` . **Abra sempre com ? simulacao=1 no final da URL** para testar sem gravar em produção (o dev server usa o mesmo Supabase); a faixa amarela confirma que o ensaio está ligado. Feche o `cloudflared` ao terminar — enquanto ele roda, qualquer pessoa com a URL vê a tela de login.

11. Convenções de desenvolvimento

- **Sem TypeScript, sem Tailwind, sem libs de UI.** Manter assim — a graça é zero configuração e zero risco de quebra em bump de versão.
 - **Regra nova** → `core/`. Se dá pra escrever como "recebe dados, devolve dados", não vai na tela. `core/` e `data/` são puros e testáveis isolados.
 - **I/O só em `repositorio.js`.**
 - **Comentários em português**, densos onde a regra é sutil (ver o cabeçalho de `rachao.js`). Explicar o *porquê*, não o *o quê*.
 - **5ª tela?** Arquivo novo em `telas/`, importa de `core / components / theme`, e `App.jsx` ganha uma linha de rota.
 - **Estado que precisa sobreviver a um F5 no meio do jogo, mas não sincronizar:** `localStorage` com `try/catch` em toda leitura/escrita (modo privado, cota).
 - **Bloco de Lances:** sempre `<LimiteErro>` + `try/catch`; nunca deixar um erro de câmera derrubar a súmula/fila.
 - **Testes:** `testes/*.teste.mjs`, sem framework, rodados com `npx vite-node testes/<arquivo>` (saem com código 1 se algo falhar). Hoje cobrem a Copa Hendor. Regra nova em `core/` merece teste ali.
 - **Testar sem gravar em produção:** `?simulacao=1` (seção 5).
 - Migração SQL: ver seção 9.2.
-

12. Limitações conhecidas e dívidas técnicas

Item	Detalhe
Escrita concorrente na base	"salva tudo" com last-write-wins. Dois organizadores editando ao mesmo tempo → um sobrescreve o outro (o Realtime avisa, mas não faz merge). Na prática há um organizador ativo por vez. A Copa Hendor cai em datas FIFA, sem rodada no mesmo dia, o que reduz o risco.
Rachão não tem histórico	encerra o dia = descarta tudo. Não há registro entre dias nem no Supabase. De propósito, mas limita relatórios.
Bundle de ~650 KB	um chunk só, sem code-splitting. Aceitável hoje; se crescer, <code>manualChunks</code> ou <code>import()</code> dinâmico.
Lances: ângulo duplicado	ver seção 8.1.
dist/ versionado	gera diff-noise; conviver com o <code>git checkout -- dist/</code> .
Poucos testes	Só <code>testes/copaHendor.teste.mjs</code> e <code>testes/rachao.teste.mjs</code> (vite-node, sem runner: <code>npx vite-node testes/<arquivo></code>). O resto do <code>core/</code> é puro justamente pra permitir testes, mas ainda não tem suíte.
Copa: sem cadastro de duplas nem sorteio	O app não sorteia duplas, inscreve jogadores nem faz repescagem (Arts. 43–47): a Copa 2026 vem de <code>data/copaHendor2026.js</code> . Uma edição nova exige cadastrar o chaveamento (hoje, em código).
Copa: uma edição na tela	<code>copaDaTemporada</code> mostra a Copa do ano de <code>base.temporada</code> (ou a última). Não há seletor de edições anteriores.
Copa: correções limitadas	Só dá para desfazer o último chute; trocas e W.O. só se desfazem antes da disputa começar; o placar das fases já jogadas é corrigível, mas chutes já lançados não são editáveis um a um.
Copa: zona da Supercopa	Mantida como no Campeonato (12 de linha + 2 goleiros; campeão fora do corte entra e empurra o último). O Art. 58 §3 admite leitura em que, se o campeão já está dentro, a vaga vai para o próximo colocado.
npm run dev usa o banco de produção	Use <code>?simulacao=1</code> para testar sem gravar. Um teste sem a flag grava de verdade.
naLinha em rodada	campo legado no shape, não usado no cálculo atual.